is `std::memory_order_release`, then `failure_order` is `std::memory_order_relaxed`.

Note

This is an atomic read-modify-write operation for the memory location comprising `*this` if the result is `true`, with memory ordering `success_order`; otherwise, it's an atomic load operation for the memory location comprising `*this` with memory ordering `failure_order`.

Std::Atomic_Compare_Exchange_Strong Nonmember Function

Atomically compares the value to an expected value and stores a new value if the values are equal. If the values aren't equal, updates the expected value with the value read.

Declaration

```
template<typename BaseType>
bool atomic_compare_exchange_strong(
    volatile atomic<BaseType>* p,BaseType * old_value,BaseType
new_value)
    noexcept;
template<typename BaseType>
bool atomic_compare_exchange_strong(
    atomic<BaseType>* p,BaseType * old_value,BaseType new_value)
noexcept;
```

Effects

```
return p->compare_exchange_strong(*old_value,new_value);
```

Std::Atomic_Compare_Exchange_Strong_Explicit Nonmember Function

Atomically compares the value to an expected value and stores a new value if the values are equal. If the values aren't equal, updates the expected value